TOWER MERGE DEFENSE

Rapport de projet — Python / Pygame

Laura Deruwez

Sommaire :

1. Présentation générale
2. Présentation des difficultés
    1. Le genre tower-defense
    2. La mécanique de fusion (partie merge)
    3. Problèmes techniques à résoudre
3. Architecture et choix techniques
    1. Architecture MVC
    2. Calcul de chemin : BFS sur la grille
    3. Déplacement continu et delta time
    4. Système de projectiles
    5. Mécaniques de fusion
    6. Gestion des états de jeu
    7. Chargement des niveaux et persistance des scores
4. Points clés algorithmiques
    1. BFS et reconstruction du chemin
    2. Interpolation linéaire pour le déplacement fluide
    3. Détection de collision projectile-cible par seuil de distance
    4. Progression des statistiques par niveau

1. Présentation générale

Mon projet est un jeu de tower-defense développé en Python avec Pygame. L'objectif est de défendre une base contre des vagues de monstres en plaçant des unités sur une grille, en les fusionnant pour les améliorer, et en survivant à toutes les vagues pour remporter la victoire.

Le projet repose sur une architecture Modèle-Vue-Contrôleur (MVC) qui structure l'ensemble du code en trois couches différentes : la partie model gère les données et la logique du jeu, la partie view se sert de Pygame pour le rendu graphique, et la partie controller gère la gestion des évènements et les mises à jour de l'état du jeu.

Pour la gestion des données, il y a un module data/ (chargement des niveaux et persistance des scores) et un fichier settings.py centralisant toutes les constantes du jeu (dimensions, couleurs, paramètres de gameplay…).

Pour exécuter le jeu, il faut exécuter main.py. Main.py va lancer la boucle de jeu principale et instancier un objet Game.

2. Présentation des difficultés

2.1 Le genre tower-defense

Un tower-defense est un jeu dans lequel des ennemis suivent un chemin prédéfini et tentent d'atteindre une zone protégée. Le joueur place des défenses (tours) sur le terrain pour les éliminer avant qu'ils n'arrivent. Les contraintes classiques du genre sont :

- Le chemin des ennemis est fixe et connu à l'avance.
- Les tours ne peuvent être placées que sur des cases spécifiques.
- La progression repose sur des vagues d'ennemis de difficulté croissante.
- La ressource principale (l'or) contraint les décisions du joueur.

2.2 La mécanique de fusion (partie merge)

Le projet introduit une mécanique de fusion inspirée des jeux de type "merge" : deux unités du même type et du même niveau peuvent être fusionnées pour former une unité de niveau supérieur. Chaque niveau de fusion double les dégâts et la vitesse d'attaque, et multiplie la portée par 1,5. Cette mécanique ajoute une dimension stratégique : il faut arbitrer entre poser de nombreuses unités faibles ou investir pour en obtenir quelques-unes très puissantes.

2.3 Problèmes techniques à résoudre

La réalisation d'un tel jeu impose plusieurs défis algorithmiques et techniques :

a) Calcul de chemin : déterminer le chemin emprunté par les monstres sur la grille qui supporte des configurations variées (une ou plusieurs entrées/sorties).

b) Déplacement continu des monstres : faire avancer les ennemis de manière fluide entre les cases de la grille et non case par case de manière saccadée.

c) Gestion du temps de jeu : découpler la logique de simulation du rendu graphique via un delta-time, et gérer des vitesses de jeu variables.

d) Projectiles : calculer la trajectoire d'un projectile vers une cible en mouvement et détecter la collision. Il fallait aussi que la trajectoire du projectile soit fluide visuellement.

e) Gestion des états de jeu : naviguer proprement entre les écrans (menu principal, jeu, pause, victoire, défaite, scoreboard) sans effets de bord.

3. Architecture et choix techniques

3.1 Architecture MVC

L'application du patron MVC est la décision structurelle la plus importante du projet. Elle garantit que :

- Le modèle (GameState, Grid, Wave, Monster, Unit…) est totalement indépendant de Pygame. Il peut être testé ou modifié sans toucher au rendu.
- Les vues (HudView, GridView, UnitView…) ne contiennent aucune logique métier. Elles reçoivent des données en paramètre et les affichent.
- Les contrôleurs (GameController, WaveController) font le lien : ils reçoivent les événements, mettent à jour le modèle, et n'écrivent jamais directement sur l'écran.

La classe Game (game.py) joue le rôle de coordinateur de haut niveau : elle gère les transitions entre écrans, délègue les événements aux bons handlers, et appelle le rendu à chaque frame. Ce n’est donc pas une architecture MVC pure mais s’en rapproche beaucoup.

Figure 1 - Diagramme de dépendances entre les modules

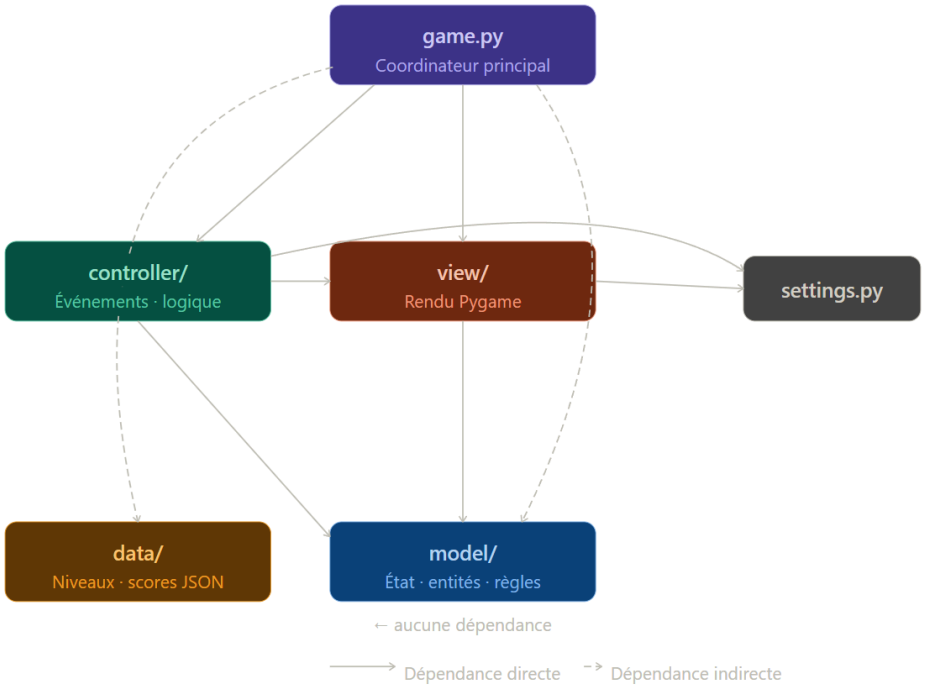


Image générée par IA, après lui avoir fourni les bulles et leur contenu, ainsi que les dépendances entre elles.

3.2 Calcul de chemin : BFS sur la grille

Le calcul du chemin est assuré par PathFinder (controller/path_finder.py) via un algorithme de recherche Breadth-First Search. Le BFS est le choix naturel ici : la grille est un graphe non pondéré (toutes les cases PATH ont le même coût), et le BFS est optimisé dans ce cas là.

L'algorithme visite les cases de type PATH adjacentes (4-connectivité), mémorise le parent de chaque case visitée dans un dictionnaire, puis reconstruit le chemin en remontant de l'arrivée au départ.

La méthode calculate_paths() gère les trois configurations utilisées :

- 1 entrée / 1 sortie → 1 chemin
- 1 entrée / 2 sorties → 2 chemins (monstres répartis aléatoirement)
- 2 entrées / 2 sorties → 2 chemins appariés

Les chemins sont calculés une seule fois au chargement du niveau. Les monstres et les projectiles les utilisent ensuite comme référence en lecture seule.

Figure 2 - Exemple de chemin avec BFS

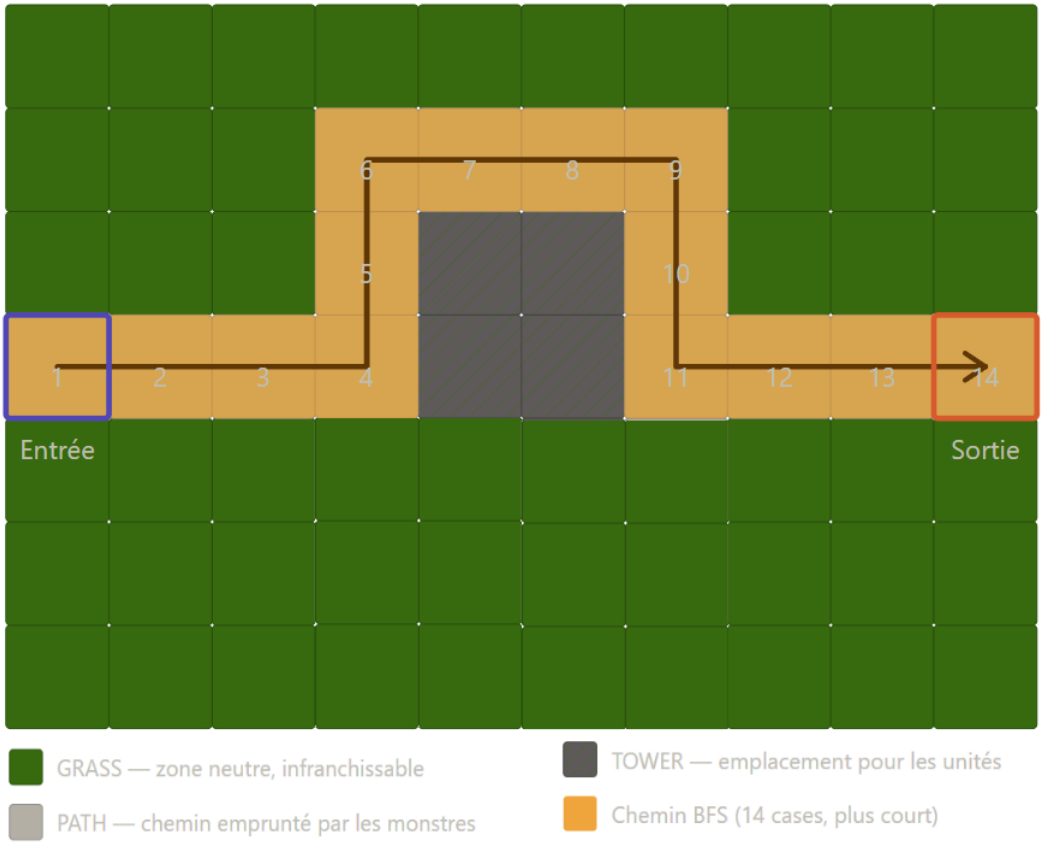


Image partiellement générée par IA, retouchée avec PAINT

3.3 Déplacement continu et delta-time

Les monstres ne se déplacent pas case par case mais de manière continue, grâce à un attribut progress (float) qui représente leur position le long du chemin. À chaque frame, progress est incrémenté de speed * dt (vitesse × delta-time).

La position en pixels est ensuite calculée entre la case courante (indice int(progress)) et la suivante (indice + 1), pondérée par la partie fractionnaire de progress. Ce calcul est partagé entre UnitView (affichage des monstres) et Projectile (ciblage).

L'usage du delta-time dans toute la logique de mise à jour (monster.move, projectile.update, wave spawn timer, auto-wave timer) garantit que le jeu tourne à la même vitesse quels que soient les FPS. La vitesse de jeu (x0,5 / x1 / x1,5) est appliquée en multipliant le dt avant de le passer au GameController, ce qui accélère ou ralentit uniformément tout le jeu.

3.4 Système de projectiles

Chaque unité a un cooldown d'attaque (_attack_cooldown). Quand il atteint zéro et qu'un monstre est à portée, elle génère un objet Projectile ciblant ce monstre. Le projectile suit sa cible en temps réel : à chaque frame, il recalcule la position en pixels du monstre (via le mécanisme décrit plus haut) et se déplace vers elle à vitesse constante : (PROJECTILE_SPEED px/s).

La collision est détectée par seuil de distance : si la distance restante est inférieure au déplacement de cette frame, le projectile "touche" sa cible, lui inflige ses dégâts et se désactive.

Les projectiles inactifs sont nettoyés en fin de frame par une liste par compréhension, et l'or du joueur n'est distribué qu'à ce moment-là (après confirmation que le monstre est bien mort), ce qui évite les doublons.

3.5 Mécanique de fusion

La fusion est gérée par MergeManager, qui contient deux méthodes :

- can_merge(u1, u2) : vérifie que les deux unités sont du même type, du même niveau, et sous le niveau maximum (MAX_UNIT_LEVEL = 5).
- merge(u1, u2) : appelle level_up() sur u1 et signale au controller que u2 doit être retirée de la grille.

La méthode level_up() dans Unit recalcule les statistiques par multiplication directe (×2 pour dégâts, ×1.5 pour portée et vitesse). Cela garantit que deux unités distinctes de niveau n feront moins de dégâts qu'une unité de niveau n+1 seule.
Le système peut être étendu à d'autres types d'unités si on en rajoute. Cela dit, il faudrait créer le type d'unité avec ses valeurs, et gérer l'affichage…

3.6 Gestion des états de jeu

L'état courant de l'application est modélisé à deux niveaux :

- AppStatus (MAP ou PLAYING) : détermine quel écran est actif.
- GameStatus (PLAYING, MENU, INFO, GAME_OVER, VICTORY) : état de la partie en cours

GameState contient des propriétés booléennes (playing, menu, info, victory, game_over) qui permettent de gérer les états de la partie et agir en conséquence. Les transitions entre deux états sont contenues dans des méthodes (toggle_menu, toggle_info, set_victory…), ce qui permet l'encapsulation des données et éviter les interventions accidentelles extérieures.

3.7 Chargement des niveaux et persistance des scores

Les niveaux sont définis dans un fichier JSON (data/level.json) contenant pour chaque niveau : la grille (matrice d'entiers), les positions d'entrée(s) et de sortie(s), et la définition des vagues (type de monstres, quantité, intervalle de spawn). Cette approche permet d'ajouter ou de modifier des niveaux sans toucher au code.

Les scores sont également chargés en JSON (data/scores.json). La fonction add_score() maintient un top 3 par niveau, trié par score décroissant.

4. Points clés algorithmiques

4.1 BFS et reconstruction du chemin

La complexité du BFS est O(V + E) où V est le nombre de cases PATH et E le nombre d'arêtes (adjacentes 4-connectées). En pratique, sur une grille 16×10 = 160 cases, c'est négligeable et n'est exécuté qu'une fois par niveau.

La reconstruction utilise un dictionnaire visited qui lie chaque case à son prédécesseur. Remonter ce dictionnaire depuis la case d'arrivée jusqu'à None donne le chemin en ordre inverse, inversé ensuite par path.reverse().

4.2 Interpolation linéaire pour le déplacement fluide

Soit progress = i + f (i entier, 0 ≤ f < 1).
La position en pixels (px, py) du monstre est :

- px = path[i].pixel_x + (path[i+1].pixel_x - path[i].pixel_x) * f + CELL_SIZE/2
- py = path[i].pixel_y + (path[i+1].pixel_y - path[i].pixel_y) * f + CELL_SIZE/2

Cette même formule est utilisée à la fois dans UnitView._draw_monster() et dans Projectile._get_target_pixel(), assurant une cohérence parfaite entre l'affichage de la cible et le ciblage du projectile.

4.3 Détection de collision projectile-cible par seuil de distance

À chaque frame, le projectile se déplace de PROJECTILE_SPEED * dt pixels. Si la distance euclidienne au point cible est inférieure à ce déplacement, le projectile est considéré comme ayant atteint sa cible.
Cela évite d'avoir à tester une intersection exacte, qui serait fragile avec des cibles en mouvement.

4.4 Progression des statistiques par niveaux

L'initialisation d'une unité au niveau 1 avec les stats de base, et la progression par level_up(), forment une suite géométrique :

- dégâts(n) = dégâts_base × 2^(n-1)
- vitesse(n) = vitesse_base × 2^(n-1)
- portée(n) = portée_base × 1.5^(n-1)

Une unité de niveau 5 a donc 16× les dégâts et la vitesse d'une unité de niveau 1 (2^4 = 16), et environ 5× sa portée. Mais le réel avantage n'est pas le nombre de dégâts par attaque mais la vitesse d'attaque qui augmente aussi. Cela crée une courbe de puissance exponentielle qui rend la fusion très récompensante.

Conclusion

Tower Merge Defense implémente le type de jeu tower-defense, mais enrichie par une mécanique de fusion originale qui ajoute une couche stratégique significative. L'architecture MVC adoptée a permis de maintenir une séparation claire des responsabilités tout au long du développement, rendant le code plus facilement extensible, modifiable et plus simple à déboguer.

Les choix algorithmiques sont adaptés aux contraintes du problème : le BFS est suffisant et optimal pour un graphe non pondéré, l'interpolation linéaire offre une fluidité visuelle sans coût de calcul significatif, et la gestion par delta-time assure une simulation déterministe indépendante du matériel.

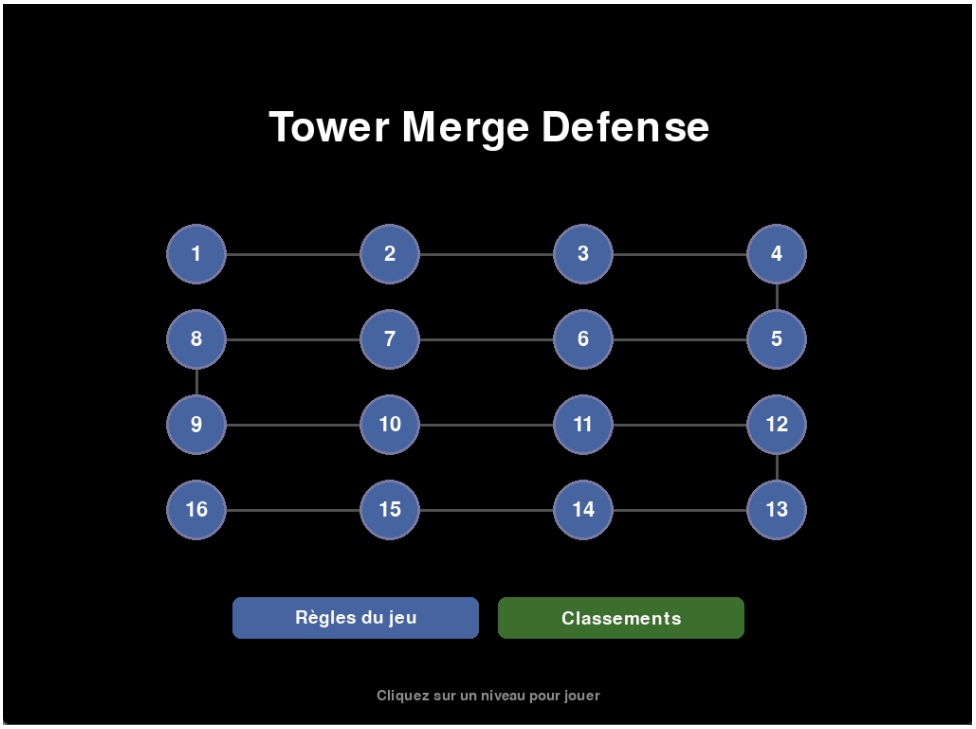
Les pistes d'évolution naturelles seraient l'ajout de nouveaux types d'unités ou de monstres (le système d'héritage s'y prête directement), la gestion de chemins multiples plus complexes, ou l'introduction d'effets de statut (ralentissement, poison) qui enrichiraient encore la dimension tactique du jeu.

Illustrations

Les diagrammes de classe, ne pouvant être affichés ici avec une qualité suffisante, sont dans le dossier diagrammes du projet rendu. Ils ont été générés avec de l'intelligence artificielle pour que ce soit plus propre que dessiné à la main.

Voici maintenant des illustrations du jeu :

- Carte des niveaux



- Ce qui est affiché si on clique sur Règles du jeu

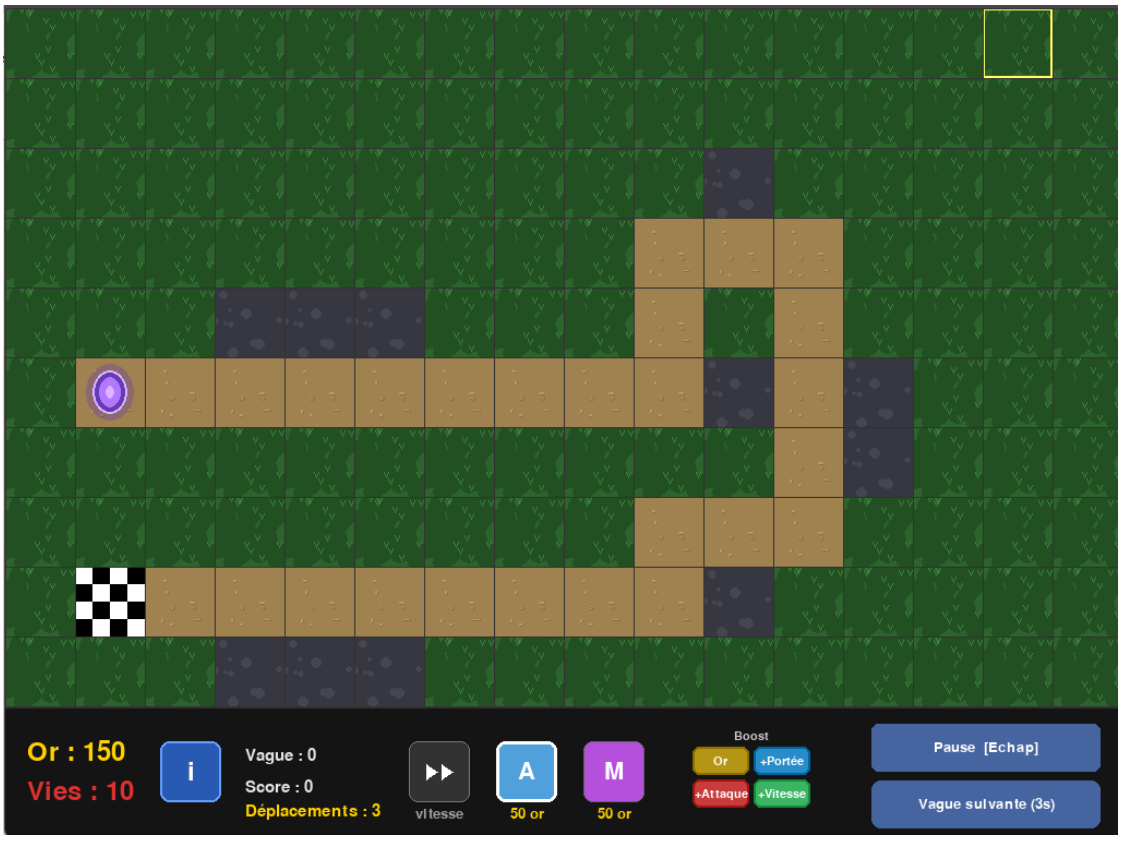


- Ce qui est affiché si on clique sur Classements

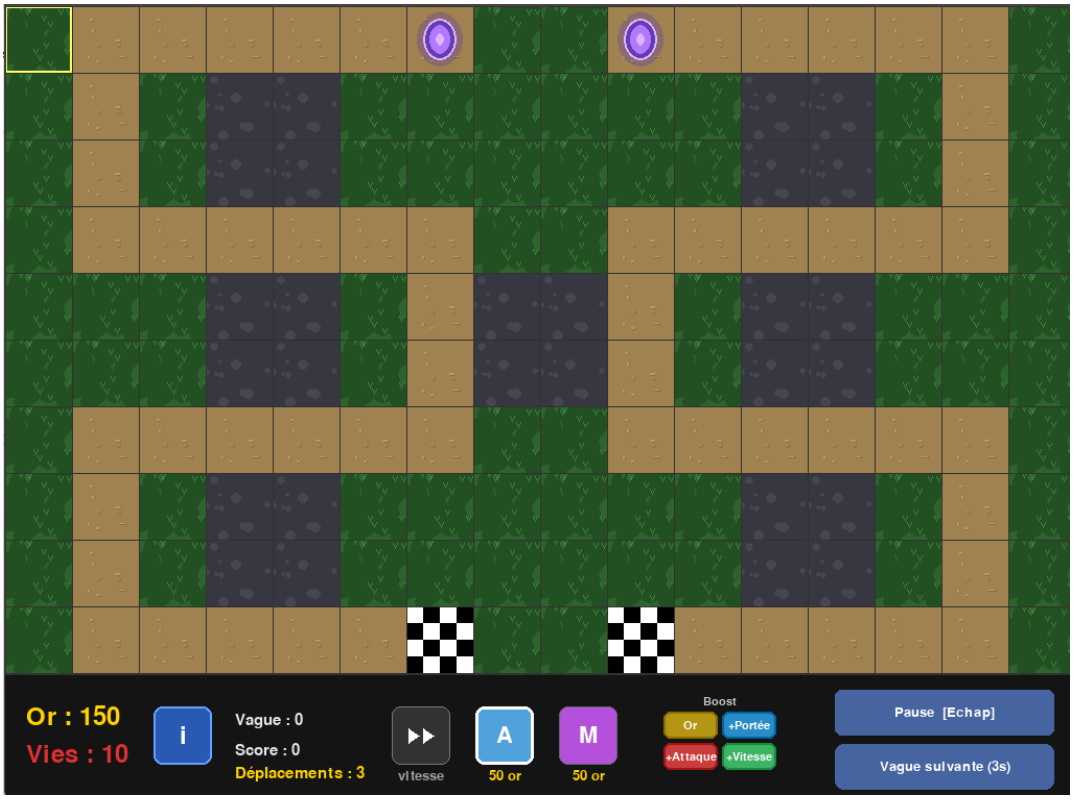


Voici maintenant des exemples de niveaux de jeu :

- Niveau 1



- Niveau 15



En jeu, le bouton i que l'on voit donne les informations sur les vagues à suivre :

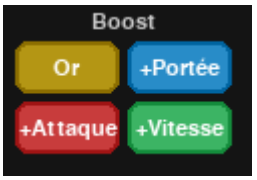


Le bouton “vitesse” a trois états et permet de ralentir/accélérer le jeu.

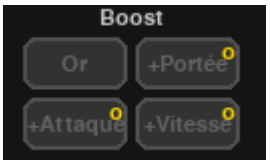


Les boosters ont une indication visuelle de leur utilisation

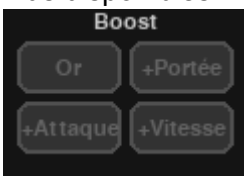
Non utilisés



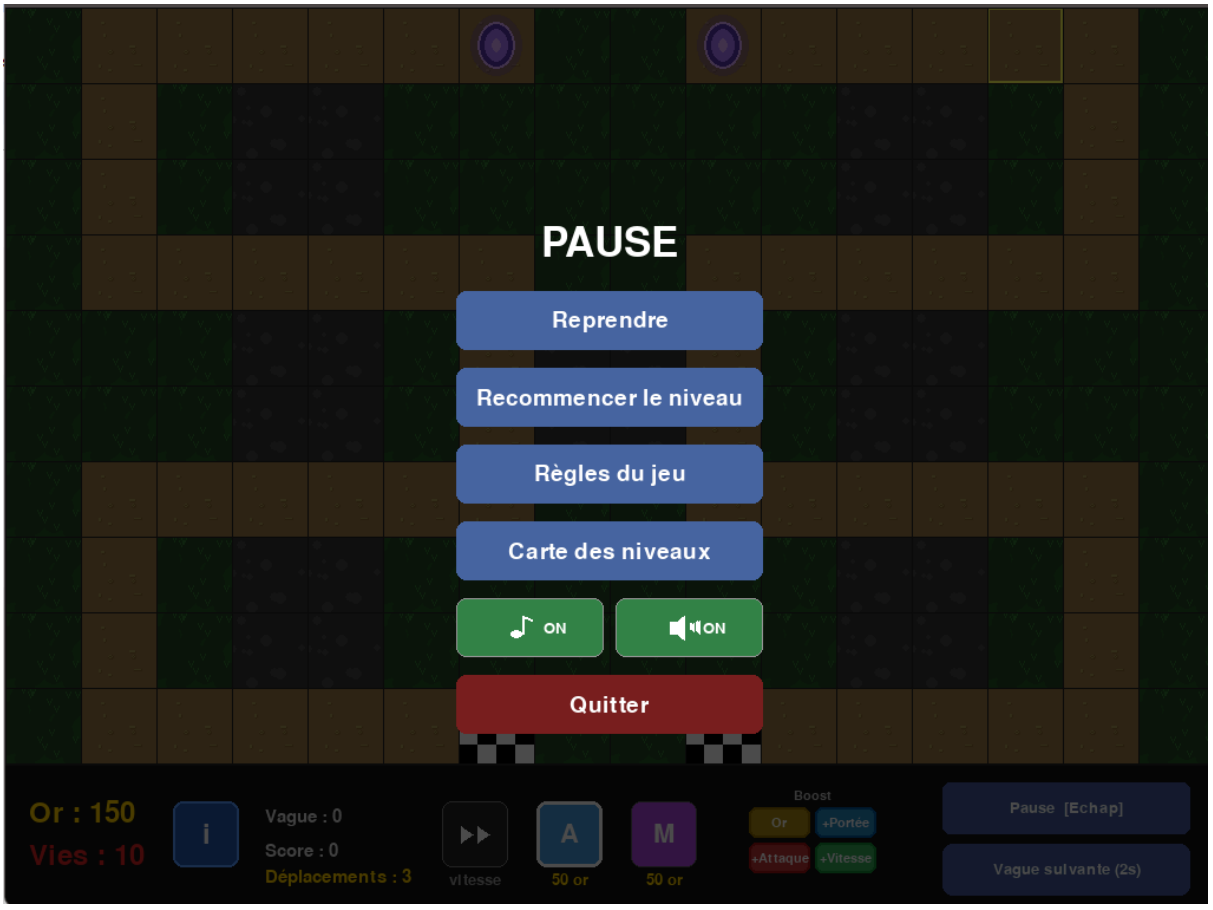
En cours d’utilisation



Plus disponibles



Le bouton Pause, ou Echap, ouvre le menu.



Le menu propose de reprendre le niveau où on en est, de recommencer le niveau, de retourner à la carte des niveaux, d’activer ou désactiver les sons et musiques, quitter le jeu et afficher les règles du jeu.
Les boutons musique et son changent d’apparence selon si ils sont activés ou non.



Les règles du jeu affichées sont les mêmes que sur la carte des niveaux (montrées plus haut).

En cas de victoire à un niveau, un écran avec les données du niveau s'affiche, comme ici :



Voilà ce qui s'affiche si on souhaite afficher les classements :



Et en cas de défaite, un écran s’affiche avec des conseils et propose de recommencer :

